



PROGRAMAÇÃO WEB 2 · AULA 04

Primeira página da aplicação

Hoje vamos continuar aprendendo...

Emanuel Nascente · Escola Técnica Mesquita

REACT :D



Montando o projeto

01

Começo

Vamos começar com a importação do Vite para o nosso projeto usando o comando abaixo, na pasta do projeto que estamos criando:

```
$ npm create vite@latest  
  nomeDoNossoApp -- --template react
```

Troque "nomeDoNossoApp" pelo nome do aplicativo como desejar.

02

Iniciando

Acesse a pasta pelo próprio prompt usando `cd` e o nome da pasta que acabamos de criar. Depois execute os dois comandos abaixo, caso ainda não tenha iniciado o serviço do React:

```
$ npm install  
$ npm run dev
```

03

Vínculo e início

Agora vamos fazer o vínculo do nosso projeto com o Visual Studio Code e, depois disso, vamos criar um novo arquivo chamado `Login.jsx` — e é nele que vamos trabalhar.

```
$ Login.jsx
```

Trabalhamos em .JSX, não em .JS

Nós trabalhamos no arquivo **.JSX**, não **.JS**.

Trabalhando em JSX nós usamos uma "extensão" do HTML dentro do nosso arquivo JS. Dessa forma temos uma sintaxe um pouco diferente do JavaScript normal.

Na prática: escrevemos marcação parecida com HTML no meio do código JavaScript, e o React se encarrega de transformar isso na tela.

JavaScript

.js

+

HTML

<tags>

=

JSX

.jsx

import

Login.jsx

```
1 import { useState } from "react";
2
3 function Login() {
4   const [email, setEmail] = useState("");
5   const [senha, setSenha] = useState("");
6   const [erro, setErro] = useState("");
7
8   ...
```

O que basicamente o `import { useState } from "react"` faz é: ir até a biblioteca do REACT e chamar a função `useState`.

useState é chamado de **Hook**. Ele permite que funções "lembrem" de informações entre uma renderização e outra.

Variáveis comuns não avisam a tela quando mudam. Já com o `useState` isso se resolve: se atualizarmos um contador de 0 para 1, ele muda na tela sem a necessidade de dar um F5.

function

Login.jsx

```
1  import { useState } from "react";
2
3  function Login() {
4    const [email, setEmail] = useState("");
5    const [senha, setSenha] = useState("");
6    const [erro, setErro] = useState("");
7
8    ...
```

Já o function Login é a nossa função que será a base do sistema. Ela é como se fosse a nossa classe: tudo que vai aparecer na tela nós colocamos dentro dessa função.

O interessante — e estranho para quem vem de outras linguagens — é que podemos ANINHAR funções...

Isso é uma prática bem específica: as funções internas definem o comportamento da nossa aplicação.

const

Login.jsx

```
1  import { useState } from "react";
2
3  function Login() {
4    const [email, setEmail] = useState("");
5    const [senha, setSenha] = useState("");
6    const [erro, setErro] = useState("");
7
8    ...
```

As consts são as nossas variáveis, e o nome vem de constante.

Ela serve para declarar uma variável que, depois de receber um valor inicial, não pode ser "reatribuída".

Repare que cada estado vem em par: o valor (email) e a função que muda esse valor (setEmail).

function handleSubmit(e)

```
Login.jsx
1    ...
2
3    function handleSubmit(e) {
4        e.preventDefault();
5
6        if (!email || !senha) {
7            setErro("Preencha todos os campos.");
8            return;
9        }
10
11    ...
```

Essa é a nossa primeira função aninhada: ela está dentro da nossa função de Login.

O `e` que aparece na declaração da função é uma convenção — ele basicamente significa que ali vamos usar eventos. Poderíamos inclusive trocar o `e` por `evento`.

Esse evento terá as informações de quem disparou o evento, que tipo de evento foi, a posição do mouse e a tecla pressionada.

if / else

```
Login.jsx
1    ...
2
3    if (email === "user@email.com" &&
4        senha === "123456") {
5        alert("Login realizado com sucesso!");
6        setErro("");
7    } else {
8        setErro("E-mail ou senha incorretos.");
9    }
10   }
11
12   ...
```

As condicionais if e else funcionam da mesma maneira que vemos em outras linguagens. Nesse exemplo nós fazemos um check: se o e-mail digitado é "user@email.com" e a senha é "123456".

Caso dê "match", vai gerar um alert na página dizendo que o login foi realizado. Caso contrário, muda a mensagem de erro para "E-mail ou senha incorretos".

E assim finalizamos as nossas duas funções: a aninhada e também a de Login.

return

```
Login.jsx
1      ...
2
3      return (
4        <div className="container">
5          <form onSubmit={handleSubmit}>
6
7            <h2>Login</h2>
8
9          ...
```

O return no JS basicamente termina em uma linha só — por isso ele usa parênteses () e não chaves, como funções, métodos e classes.

Porém, se precisarmos de mais linhas, podemos escrever seguindo a sintaxe do exemplo ao lado.

E dentro do return nós vamos trabalhar com trechos em HTML e também em JS.

input

```
Login.jsx
1  ...
2
3      <input
4          type="email"
5          placeholder="Email"
6          value={email}
7          onChange={(e) =>
8              setEmail(e.target.value)}
9      />
10
11  ...
```

Quando usamos o `type="email"` nós dizemos ao navegador que esse campo é um e-mail — o teclado do teu celular vai abrir no formato de e-mail, por exemplo.

O placeholder é aquele texto cinza que aparece dentro do campo; dá para trocar por qualquer texto que for mais interessante.

O `value` junto com o `onChange` transforma o campo em um componente controlado: o React passa a ser a fonte da verdade do que está digitado.

Atenção: é o `value` **sem** o `onChange` que traria o campo como somente leitura.

{erro && ...}

```
Login.jsx
1    ...
2
3    <input
4      type="password"
5      placeholder="Senha"
6      value={senha}
7      onChange={(e) =>
8        setSenha(e.target.value)}
9    />
10   {erro && <p className="erro">{erro}</p>}
11
12   ...
```

Se fôssemos ler a última linha em voz alta, leríamos assim:

Se a variável **erro** for verdadeira (tiver algum texto), ENTÃO mostre o parágrafo **<p>** com a mensagem.

Ele usa uma lógica do JavaScript chamada Curto-Circuito para decidir se deve ou não mostrar algo na tela.

button

```
Login.jsx
1  ...
2
3  <button type="submit">Entrar</button>
4  </form>
5  </div>
6  );
7  }
8
9  export default Login;
```

É idêntico ao button do HTML, com algumas mudanças: basicamente ele vai avisar, nos eventos, que nós clicamos nesse botão.

Como ele é `type="submit"` e está dentro do `<form>`, clicar nele dispara o `onSubmit` — ou seja, o nosso `handleSubmit`.

export default Login

Essa linha no final do arquivo é o que permite que esse componente seja usado em outros lugares.

Agora vamos estilizar

A página deve ser estilizada de acordo com as tuas preferências. Porém, vamos usar nesse exemplo uma estética que acabei criando — e para usar essa estilização vamos fazer o seguinte passo a passo:

A Criar um novo arquivo dentro do nosso projeto.

B Nomear ele para **Login.css**.

C Importar ele no nosso arquivo JSX usando **import './Login.css';**

Login.css

Login.css

```
1  .container {
2    display: flex;
3    justify-content: center;
4    align-items: center;
5    height: 100vh;
6    background: #f0f2f5;
7  }
8
9  form {
10   background: white;
11   padding: 2rem;
12   border-radius: 8px;
13   box-shadow: 0 2px 10px rgba(0,0,0,0.1);
14   display: flex;
15   flex-direction: column;
16   gap: 1rem;
17   width: 300px;
18 }
```

Login.css

```
1  input {
2    padding: 0.75rem;
3    border: 1px solid #ccc;
4    border-radius: 6px;
5    font-size: 1rem;
6  }
7
8  button {
9    padding: 0.75rem;
10   background: #4f46e5;
11   color: white;
12   border: none;
13   border-radius: 6px;
14   font-size: 1rem;
15   cursor: pointer;
16 }
17
18 button:hover { background: #4338ca; }
19 .erro { color: red; font-size: 0.85rem; }
```

App.jsx

```
App.jsx
1  import Login from "../Login";
2
3  function App() {
4    return <Login />;
5  }
6
7  export default App;
```

Falta importar o Login dentro do arquivo App.jsx — é ele que o React coloca na tela quando a aplicação sobe.

O `<Login />` é o nosso componente sendo usado como se fosse uma tag do HTML. Isso só é possível por causa do **export default** lá no final do Login.jsx.

Salve tudo, volte ao navegador e a tela de login já deve estar lá.

0 teu trabalho

01 Adicionar botão **mostrar / ocultar senha**.

02 Editar a estética no CSS e deixá-la adequada ao visual que você deseja para a tua aplicação.

03 Começar a criar a nova página da tua aplicação, conforme conversaremos em aula :)